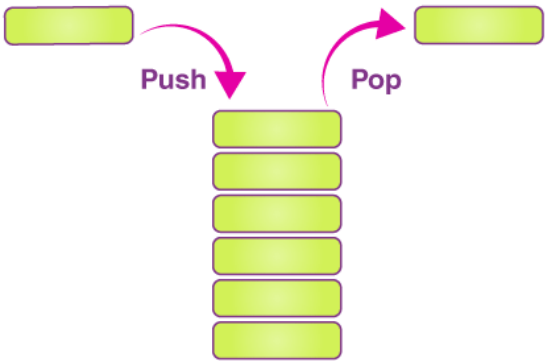
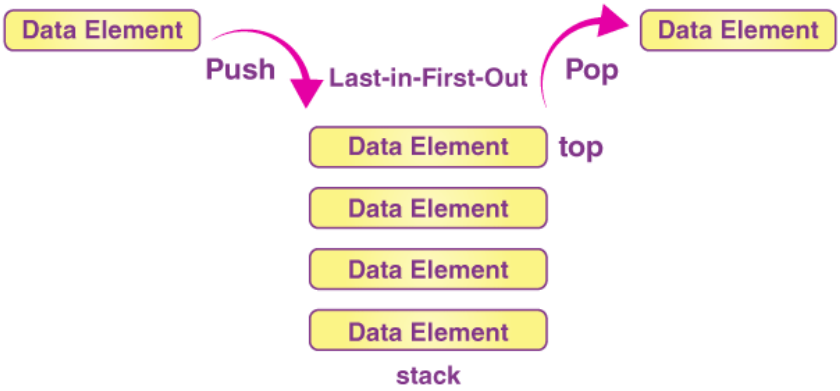
Unit-III: Stack

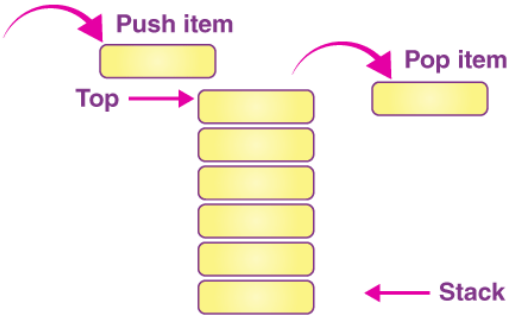
❖ What is a Stack?

A Stack is a linear data structure that holds a linear, ordered sequence of elements. It is an abstract data type. A Stack works on the LIFO process (Last In First Out), i.e., the element that was inserted last will be removed first. To implement the Stack, it is required to maintain a pointer to the top of the Stack, which is the last element to be inserted because we can access the elements only on the top of the Stack.





❖ Operation on Stack



1. **PUSH:** PUSH operation implies the insertion of a new element into a Stack. A new element is always inserted from the topmost position of the Stack; thus, we always need to check if the top is empty or not, i.e., TOP=Max-1 if this condition goes false, it means the Stack is full, and no more elements can be inserted, and even if we try to insert the element, a Stack overflow message will be displayed.

Algorithm:

Step-1: If TOP = Max-1

Print "Overflow"

Goto Step 4

Step-2: Set TOP= TOP + 1

Step-3: Set Stack[TOP]= ELEMENT

Step-4: END

2. **POP:** POP means to delete an element from the Stack. Before deleting an element, make sure to check if the Stack Top is NULL, i.e., TOP=NULL. If this condition goes true, it means the Stack is empty, and no deletion operation can be performed, and even if we try to delete, then the Stack underflow message will be generated.

Algorithm:

Step-1: If TOP= NULL

Print "Underflow"

Goto Step 4

Step-2: Set VAL= Stack[TOP]

Step-3: Set TOP= TOP-1

Step-4: END

3. **PEEK or TOP**: When we need to return the value of the topmost element of the Stack without deleting it from the Stack, the Peek operation is used. This operation first checks if the Stack is empty, i.e., TOP = NULL; if it is so, then an appropriate message will display, else the value will return.

Algorithm:

Step-1: If TOP = NULL

PRINT "Stack is Empty"

Goto Step 3

Step-2: Return Stack[TOP]

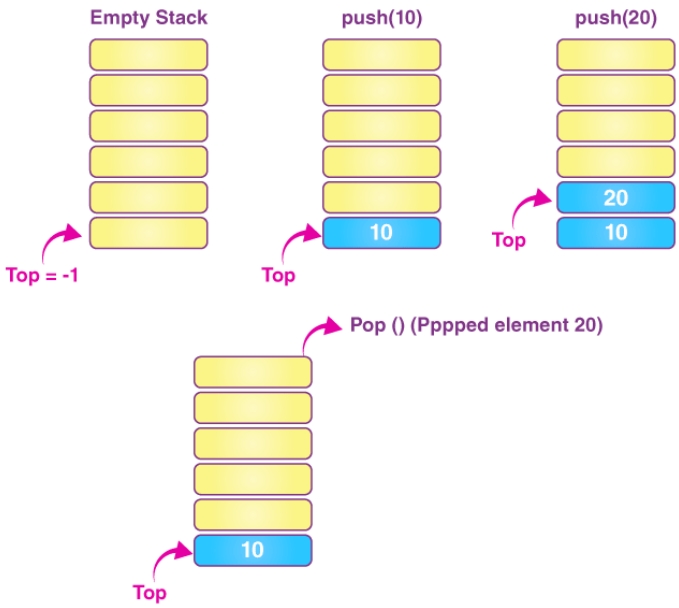
Step-3: END

4**. isEmpty():** isEmpty() operation is a boolean operation that is used to determine if the stack is empty or not. This operation will return true if the stack is empty, else false.

5**. size():** Size operation in Stack is used to return the count of elements that are present inside the stack.

- **Representation of the Stack**

A Stack can be a fixed specific size, or it can be dynamic, i.e., the Stack size can be changed dynamically. It can be represented by means of Pointer, Array, Structure, and Linked List.



Application of the Stack:

1. A Stack can be used for evaluating expressions consisting of operands and operators.
2. Stacks can be used for Backtracking, i.e., to check parenthesis matching in an expression.
3. It can also be used to convert one form of expression to another form.
4. It can be used for systematic Memory Management.

Advantages of Stack:

1. A Stack helps to manage the data in the 'Last in First out' method.
2. When the variable is not used outside the function in any program, the Stack can be used.
3. It allows you to control and handle memory allocation and deallocation.
4. It helps to automatically clean up the objects.

Disadvantages of Stack:

1. It is difficult in Stack to create many objects as it increases the risk of the Stack overflow.
2. It has very limited memory.
3. In Stack, random access is not possible.